The lpalign Package

Jamie Fravel

Published as lpalign v1.7 on July 17th 2026

jamiefravel@me.com
github.com/jfravel/lpalign

Abstract

This is a collection of macros that I originally developed while writing my dissertation but which have been through a number of iterations since. They are mostly focused on displaying, naming and referencing mathematical programming problems. The lpalign environment macro formats math programs for legibility in both code and display. The named paragraph and sub-equation macros are useful in any document which defines and references multiple problems, algorithms or multi-line environments.

Contents

1 Installation 1

2 The lpalign Environment 1
- Constraint Alignment 2
- Qualified Constraints 3
- Variable Ranges 3
- Whitespace 3
- Multiline Objectives 4
- Spanning Columns 6
- Taging Long Expressions . . . 6

3 The lpregion Environment 7
- Nesting lp Environments . . . 8

4 Named Paragraphs 10

5 Named Sub-equations 11
- Subtag Formatting 11
- Tag/Subtag Overrides 13

6 Sub-References 14

7 Combining Macros 15

8 Spacing Macros 16
- Quick Spacing 16
- Delimiter Spacing 17
- Evaluator Spacing 17

9 Package Options 18

10 Version History 19

1 Installation

Copy `lpalign.sty` to the same directory as your main `.tex` file and add the command `\usepackage{lpalign}` to your preamble. The package loads IEEEtrantools, expl3 and l3keys2e. If available, it also loads mathtools and stackengine, with small internal fallbacks for the features used here. I recommend using these macros in conjunction with the hyperref package so that custom tags/references are clickable.

2 The lpalign Environment

lpalign is an environment macro built on the IEEEeqnarray environment from IEEEtrantools. It formats mathematical programs so that they are legible in code as well as in the final pdf. Its primary functions are to automatically format and position the 'sense' (maximize or minimize) and 'subject to' terms within a gutter to the left of the body in addition to aligning constraints and qualifiers.

The lpalign environment takes two arguments—`<sense>` and `<objective>`—along with several optional keys:

`vars` – subscript on the 'sense' term. Commonly used to state variables and ranges.

`vars-style` – font style for the `vars` value. Defaults to `\scriptstyle` and may be changed globally after package load by `\setvarsstyle`; see Section 9.

`st` – text used for the 'subject to' term. Defaults to `s.t.` and may be changed during package load by `st`.

`flush` – controls the alignment behavior of the main constraint column. Takes options `l`, `c`, `r` and `a`. The default value of `a` can be changed during package load by the `flush` option.

`row-sep` – controls the spacing between constraints. The default value of `0.2em` can be changed during package load by the `row-sep` option.

`objective-sep` – controls the spacing between objective function and the first constraint as added to `row-sep`. The default value of `0em` can be changed during package load by the `objective-sep` option.

`qualifier-sep` – controls the spacing between constraint body and the qualifier (for-all) term. The default value of `2em` can be changed during package load by the `qualifier-sep` option.

`gutter-sep` – controls the spacing between 'sense'/'subject to' terms and the constraint body. The default value of `0.75em` can be changed during package load by the `gutter-sep` option.

`sense-shift` – adds a vertical shift to the 'sense' term.

`st-shift` – adds a vertical shift to the 'subject to' term.

The lpalign* environment is identical without printing equation reference tags unless explicitly called for by the `\tag` command.

```
\begin{lpalign}{Maximize}{\mathbf{c}^\top\mathbf{x}}
  A\mathbf{x} &\le \mathbf{b}  \\
   \mathbf{x} &\ge \mathbf{0}
\end{lpalign}
```

$$\begin{lpalign}{Maximize}{\mathbf{c}^\top\mathbf{x}}
  A\mathbf{x} &\le \mathbf{b} \\
   \mathbf{x} &\ge \mathbf{0}
\end{lpalign}$$

Constraint Alignment

The environment defaults to `alignat`-style behavior; the constraint rows will be aligned at the ampersand `&`.

```
Ax+s &= b+0 \\
 x,s &\ge 0
```

$$\begin{lpalign}{max}{c^\top x}
Ax+s &= b+0 \\
 x,s &\ge 0
\end{lpalign}$$

```
Ax+&s = b+0 \\
   &x,s \ge 0
```

$$\begin{lpalign}{max}{c^\top x}
Ax+&s = b+0 \\
   &x,s \ge 0
\end{lpalign}$$

The `flush` argument can be used to adjust global alignment. Options are `l`, `c` and `r` in addition to its default behavior `a`.

```
\begin{lpalign*}
[flush=l]
{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}
```

$$\begin{lpalign*}[flush=l]{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}$$

```
\begin{lpalign*}
[flush=c]
{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}
```

$$\begin{lpalign*}[flush=c]{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}$$

```
\begin{lpalign*}
[flush=r]
{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}
```

$$\begin{lpalign*}[flush=r]{max}{c^\top x}
  Ax+s = b+0 \\
    x,s \ge 0
\end{lpalign*}$$

Qualified Constraints

Qualified constraints are also possible. Another ampersand `&` will properly space and align a 'for-all' ($\forall$) statement.

```
\begin{lpalign*}{Maximize}{\sum_{j=1}^mc_jx_j}
  \mathbf{a}_i^\top\mathbf{x} &\le b_i
                    & \forall\ i\in [\![n]\!]      \\
     x_i &\ge 0     & \forall\ i\in \{1,2,...,n\}
```

$$\begin{aligned} \text{Maximize} \quad & \sum_{j=1}^{m} c_j x_j \\ \text{s.t.} \quad & \mathbf{a}_i^\top \mathbf{x} \le b_i && \forall\ i \in [\![n]\!] \\ & x_i \ge 0 && \forall\ i \in \{1,2,...,n\} \end{aligned}$$

The qualifier column still works with a centered/flushed environment body.

Variable Ranges

Subscripts can be placed under the 'sense' term via the `vars` option.

```
\begin{lpalign*}[vars=\mathbf{x}\in\mathbb{R}_+]
  {Maximize}{c_1x_1 + \dots + c_mx_m}
  A\mathbf{x} &\le \mathbf{b}
\end{lpalign*}
```

$$\begin{aligned} \underset{\mathbf{x}\in\mathbb{R}_+}{\text{Maximize}} \quad & c_1 x_1 + \cdots + c_m x_m \\ \text{s.t.} \quad & A\mathbf{x} \le \mathbf{b} \end{aligned}$$

Whitespace

The `sep` options can be used to modify the whitespace between elements. Their default values are mutable at package load.

```
\begin{lpalign}
[
objective-sep=1em,
```

```
qualifier-sep=3em,
gutter-sep=3em,
row-sep=1em,
]
{Maximize}{\sum_{j=1}^mc_jx_j}
\mathbf{a}_i^\top\mathbf{x} &\le b_i
                & \forall\ i\in [\![n]\!]     \\
\mathbf{a}_i^\top\mathbf{x} &\ge d_i
                  & \forall\ i\in [\![n]\!]     \\[3em]
   x_i &\ge 0   & \forall\ i\in \{1,2,...,n\}
\end{lpalign}
```

$$\text{Maximize} \qquad \sum_{j=1}^{m} c_j x_j \tag{4}$$

$$\text{s.t.} \qquad \mathbf{a}_i^\top \mathbf{x} \le b_i \qquad \forall\, i \in [\![n]\!] \tag{5}$$

$$\mathbf{a}_i^\top \mathbf{x} \ge d_i \qquad \forall\, i \in [\![n]\!] \tag{6}$$

$$x_i \ge 0 \qquad \forall\, i \in \{1, 2, ..., n\} \tag{7}$$

`objective-sep` is added to `row-sep` between the objective and first constraint rows. Remember that additional space can be added between individual rows (or lines of text) via an optional argument on `\\`.

Multiline Objectives

Very long objective functions can be expressed in multiple lines by supplying an explicit aligned environment from amsmath. This avoids overflow and keeps the objective aligned with the guttered 'sense' term. The same can be done within the constraints. Tags and qualifiers will be placed correctly.

```
\begin{lpalign}{Maximize}
  {\begin{aligned}
    c_1&x_1+c_2x_2+c_3x_3+c_4x_4 \\
          &+c_5x_5+...+c_mx_m
  \end{aligned}}
  \begin{aligned}
```

```
    a_{i1}&x_1+a_{i2}x_2+a_{i3}x_3 \\
          &+a_{i4}x_4+...+a_{im}x_nm
  \end{aligned} &\le b_i   & \forall\ i\in [\![n]\!]   \\
  \mathbf{x} &\ge \mathbf{0}
\end{lpalign}
```

$$\begin{aligned}
\text{Maximize} \quad & c_1 x_1 + c_2 x_2 + c_3 x_3 + c_4 x_4 \\
& + c_5 x_5 + ... + c_m x_m && && (8)\\
\text{s.t.} \quad & a_{i1} x_1 + a_{i2} x_2 + a_{i3} x_3 \\
& + a_{i4} x_4 + ... + a_{im} x_n m \le b_i && \forall\ i \in [\![n]\!] && (9)\\
& \mathbf{x} \ge \mathbf{0} && && (10)
\end{aligned}$$

The 'sense' and 'subject to' terms can be freely shifted up or down. Other amsmath-compatible wrappers or environments can be nested within the objective but math mode may need to be manually re-engaged

```
\begin{lpalign}
  [ sense-shift=0.5\baselineskip,
    st-shift=0.5\baselineskip ]
  {Maximize}
  {\fbox{$\begin{aligned}
   c_1&x_1+c_2x_2+c_3x_3+c_4x_4\\
      &+c_5x_5+...+c_mx_m
  \end{aligned}$}}
  ...
\end{lpalign}
```

$$\begin{aligned}
\text{Maximize} \quad & \boxed{\begin{aligned} & c_1 x_1 + c_2 x_2 + c_3 x_3 + c_4 x_4 \\ & + c_5 x_5 + ... + c_m x_m \end{aligned}} && && (11)\\
\text{s.t.} \quad & a_1 x_1 + a_2 x_2 + a_3 x_3 \\
& + a_4 x_4 + ... + a_m x_m \le b_i && \forall\ i \in [\![n]\!] && (12)\\
& \mathbf{x} \ge \mathbf{0} && && (13)
\end{aligned}$$

Note that, for some nested environments like `aligned` from amsmath, the optional argument `[t]` will align the sense term with the top row of the environment. Be careful when using this feature in the constraints as it will disrupt alignment with the remaining row.

Spanning Columns

The `\span` command tells `IEEEeqnarray` (and therefore `lpalign`) to ignore a column for a particular row. This is useful for letting qualifiers wrap bulky constraints.

```
\begin{lpalign}{minimize}{c_1x_2+c_2x_2+\dots+c_nx_n}
  &\sum_{j=1}^na_{ij}x_j \GE b_i   &\forall\ i\in[\![m]\!] \\
  &\mathbf{a}_i^\top\mathbf{x} \GE b_i
                                   &\forall\ i\in[\![m]\!] \\
  &\begin{bmatrix} a_{11}&a_{12}&\dots &a_{1n}\\
   a_{21}&a_{22}&\dots &a_{2n}\\ \vdots&\vdots&\ddots&     \\
   a_{m1}&a_{m2}&      &a_{mn} \end{bmatrix}
   \begin{bmatrix} x_1\\x_2\\\vdots\\x_n \end{bmatrix} \GE
   \begin{bmatrix} b_1\\b_2\\\vdots\\b_m \end{bmatrix}
  \span\span \\
  \hfill A\mathbf{x}\GE\mathbf{b} \span\span
\end{lpalign}
```

$$\begin{lpalign}{minimize}{c_1x_2+c_2x_2+\dots+c_nx_n}
  &\sum_{j=1}^na_{ij}x_j \GE b_i   &\forall\ i\in[\![m]\!] \\
  &\mathbf{a}_i^\top\mathbf{x} \GE b_i
                                   &\forall\ i\in[\![m]\!] \\
  &\begin{bmatrix} a_{11}&a_{12}&\dots &a_{1n}\\
   a_{21}&a_{22}&\dots &a_{2n}\\ \vdots&\vdots&\ddots&   \\
   a_{m1}&a_{m2}&   &a_{mn} \end{bmatrix}
   \begin{bmatrix} x_1\\x_2\\\vdots\\x_n \end{bmatrix} \GE
   \begin{bmatrix} b_1\\b_2\\\vdots\\b_m \end{bmatrix}
  \span\span \\
  \hfill A\mathbf{x}\GE\mathbf{b} \span\span
\end{lpalign}$$

An `\hfill`, two `\span`'s and one zero-width space will right-flush a row. Be careful when using `span` alongside the `flush` option since there is one fewer column.

Taging Long Expressions

If, in an emergency situation (like a two column journal), tags can be moved down a row with the macro `\downtag` which is really just a combination of `\span`, `\notag` and `\\`. Labeling/referencing should continue to work as expected. An additional `\span` will be required in the objective function of aligned environments. This is not the most robust solution, so don't rely on it too far.

```
\begin{lpalign}[row-sep=-0.2em]
{maximize}{c_1x_1+c_2x_2+c_3x_3+c_4x_4+c_5x_5
            +c_6x_6+c_7x_7+c_8x_8+c_9x_9 \span\downtag}
a_1x_1+a_2x_2+a_3x_3+a_4x_4+a_5x_5+a_6x_6
              +a_7x_7+a_8x_8 &\ge \mathbf{b} \downtag \\
x &\ge 0 \downtag
\end{lpalign}
\begin{lpalign}[flush=l, row-sep=-0.2em]
{maximize}{c_1x_1+c_2x_2+c_3x_3+c_4x_4+c_5x_5
                  +c_6x_6+c_7x_7+c_8x_8+c_9x_9 \downtag}
a_1x_1+a_2x_2+a_3x_3+a_4x_4+a_5x_5+a_6x_6
              +a_7x_7+a_8x_8 \ge \mathbf{b} \downtag \\
x \ge 0 &\forall\ x
\end{lpalign}
```

$$\begin{aligned}
\text{maximize}\quad & c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 + c_5x_5 + c_6x_6 + c_7x_7 + c_8x_8 + c_9x_9 && (19)\\
\text{s.t.}\quad & a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4 + a_5x_5 + a_6x_6 + a_7x_7 + a_8x_8 \ge \mathbf{b} && (20)\\
& x \ge 0 && (21)
\end{aligned}$$

$$\begin{aligned}
\text{maximize}\quad & c_1x_1 + c_2x_2 + c_3x_3 + c_4x_4 + c_5x_5 + c_6x_6 + c_7x_7 + c_8x_8 + c_9x_9 && (22)\\
\text{s.t.}\quad & a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4 + a_5x_5 + a_6x_6 + a_7x_7 + a_8x_8 \ge \mathbf{b} && (23)\\
& x \ge 0 \qquad \forall\ x && (24)
\end{aligned}$$

3 The lpregion Environment

The lpregion environment mirrors the formatting features of lpalign without objective, sense or s.t. terms for the purpose of describing regions only. It takes only a few optional keys:

`flush` – controls the alignment behavior of the main constraint column. Takes options `l`, `c`, `r` and `a`. The default value of `a` can be changed during package load by the `region-flush` option.

`row-sep` – controls the spacing between constraints. The default value of `0.2em` can be changed during package load by the `row-sep` option.

`qualifier-sep` – controls the spacing between constraint body and the qualifier (for-all) term. The default value of `2em` can be changed during package load by the `qualifier-sep` option.

The lpregion* environment is identical without printing equation reference tags unless explicitly called for by the \tag command.

```
\begin{lpregion*}
  &\sum_{j=1}^na_{ij}x_j \GE b_i   &\forall\ i\in[\![m]\!] \\
  &\mathbf{a}_i^\top\mathbf{x} \GE b_i
                                   &\forall\ i\in[\![m]\!] \\
  &\begin{bmatrix} a_{11}&a_{12}&\dots &a_{1n}\\
   a_{21}&a_{22}&\dots &a_{2n}\\ \vdots&\vdots&\ddots&     \\
   a_{m1}&a_{m2}&      &a_{mn} \end{bmatrix}
   \begin{bmatrix} x_1\\x_2\\\vdots\\x_n \end{bmatrix} \GE
   \begin{bmatrix} b_1\\b_2\\\vdots\\b_m \end{bmatrix}
  \hfill A\mathbf{x}\GE\mathbf{b} \span\span\hspace{0pt}
\end{lpregion*}
```

$$\sum_{j=1}^{n} a_{ij}x_j \;\geq\; b_i \qquad \forall\; i \in [\![m]\!]$$

$$\mathbf{a}_i^\top \mathbf{x} \;\geq\; b_i \qquad \forall\; i \in [\![m]\!]$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \\ a_{m1} & a_{m2} & & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \;\geq\; \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

$$A\mathbf{x} \;\geq\; \mathbf{b}$$

$$\sum_{j=1}^{n} a_{ij}x_j \;\geq\; b_i \qquad \forall\; i \in [\![m]\!]$$

$$\mathbf{a}_i^\top \mathbf{x} \;\geq\; b_i \qquad \forall\; i \in [\![m]\!]$$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \\ a_{m1} & a_{m2} & & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \;\geq\; \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

$$A\mathbf{x} \;\geq\; \mathbf{b}$$

Nesting lp Environments

You may want to use these formatting conventions nested within another environment. The `lpregionbox` and `lpalignbox` environments can be nested. Maybe you want to define a polyhedron.

```
\begin{equation*}
P =
\left\lbrace
x\in\mathbb{R}^n \ \middle\vert\
  \begin{lpregionbox}[flush=l]
  \sum_{j=1}^na_{ij}x_j \GE b_i
                &\forall\ i\in[\![m]\!] \\
  \mathbf{a}_i^\top\mathbf{x} \GE b_i
                &\forall\ i\in[\![m]\!]
  \end{lpregionbox}
\right\rbrace
\end{equation*}
```

$$P = \left\{ x \in \mathbb{R}^n \;\middle|\; \begin{array}{ll} \displaystyle\sum_{j=1}^{n} a_{ij} x_j \;\geq\; b_i & \forall\, i \in [\![m]\!] \\ \mathbf{a}_i^\top \mathbf{x} \;\geq\; b_i & \forall\, i \in [\![m]\!] \end{array} \right\}$$

If wrapped delimiters appear short or vertical centering is... off, then try setting `\delimitershortfall=0pt` or adding a phantom, vertical character like `\vphantom{\int}` to the problem row.

```
\begin{equation*}
P =
\left\lbrace
x\in\mathbb{R}^n \ \middle\vert\
  \begin{lpregionbox}[flush=l]
  \sum_{j=1}^na_{ij}x_j \GE b_i
                &\forall\ i\in[\![m]\!] \\
  \mathbf{a}_i^\top\mathbf{x} \GE b_i
                &\forall\ i\in[\![m]\!] \vphantom{\int}
  \end{lpregionbox}
\right\rbrace
\end{equation*}
```

$$P = \left\{ x \in \mathbb{R}^n \;\middle|\; \begin{array}{ll} \displaystyle\sum_{j=1}^{n} a_{ij} x_j \;\geq\; b_i & \forall\, i \in [\![m]\!] \\ \mathbf{a}_i^\top \mathbf{x} \;\geq\; b_i & \forall\, i \in [\![m]\!] \end{array} \right\}$$

Similarly, the `lpalignbox` environment is a nestable version of `lpalign`. Maybe you have a complicated, minimax problem

```
\begin{lpalign}{min}{y}
y \left[
  \begin{lpalignbox}[x]
  {max}{\mathbf{c}^\top\mathbf{x}}
  \mathbf{a}_i^\top\mathbf{x} &\ge b_i
            &\forall\ i\in[\![m]\!] \\
  \mathbf{x} &\ge 0
  \end{lpalignbox}
\right] &\le d \\
y &\ge 0
\end{lpalign}
```

$$\begin{aligned}
\min \quad & y && && (25)\\
\text{s.t.} \quad & y \left[\begin{array}{lll} \max\limits_{x} & \mathbf{c}^\top\mathbf{x} & \\ \text{s.t.} & \mathbf{a}_i^\top\mathbf{x} \ge b_i & \forall\ i \in [\![m]\!] \\ & \mathbf{x} \ge 0 & \end{array}\right] \le d && (26)\\
& && y \ge 0 & (27)
\end{aligned}$$

4 Named Paragraphs

The `namedpara` command displays a paragraph marker that takes an abbreviation, which is useful for referencing specific problems or conventions.

```
\namedpara{Named Paragraph}[NPg]\label{npara1} \\
Paragraph \ref{npara1} is a named paragraph.
```

Named Paragraph (NPg)
Paragraph NPg is a named paragraph.

Currently supported optional keys include

`abv` – abbreviation used when referencing. A single argument with no `=` sign is also interpreted as this abbreviation.

`indent` – indent distance. The default value of `2em` can be changed during package load by the `namedpara-indent` option; see Section 9.

`before` – arbitrary code placed between the indent and the title.

`after` – arbitrary code placed after the abbreviation.

`flush` – centers or right-flushes the paragraph title via `center` or `right`.

```
\namedpara{Indented}[abv=IND, indent=6em]

\namedpara{Surrounded}
          [abv=SUR, flush=center, before=$>>>$, after=$<<<$]

\namedpara{Rightflushed}[abv=RIGHT, flush=right]
```

Indented (IND)

>>>**Surrounded (SUR)**<<<

Rightflushed (RIGHT)

5 Named Sub-equations

The `namedsubeqs` environment uses a given name as the first index in any enclosed equation reference tag. A second index is appended as in a traditional `subequations` environment.

```
\begin{namedsubeqs}{EQ}
\begin{align}
  A\mathbf{x} = \mathbf{b}   \label{eq:1}   \\
  C\mathbf{y} = \mathbf{d}   \label{eq:2}
\end{align}
\end{namedsubeqs}

Equations \ref{eq:1} and \ref{eq:2} are equations.
```

$$A\mathbf{x} = \mathbf{b} \quad \text{(EQ.a)}$$
$$C\mathbf{y} = \mathbf{d} \quad \text{(EQ.b)}$$

Equations EQ.a and EQ.b are equations.

Subtag Formatting

An optional argument allows you to change the style of the sub-tag. The currently supported keys are `style` for the second-index formatter (default `\alph`) and `punct` for the separator between the main tag and the sub-tag (default `.`).

```
\begin{namedsubeqs}{EQ}[style=\roman]
                \begin{namedsubeqs}{EQ}[style=\arabic]
```

$$A\mathbf{x} = \mathbf{b} \quad \text{(EQ.i)} \qquad A\mathbf{x} = \mathbf{b} \quad \text{(EQ.1)}$$
$$C\mathbf{y} = \mathbf{d} \quad \text{(EQ.ii)} \qquad C\mathbf{y} = \mathbf{d} \quad \text{(EQ.2)}$$

Fonts can be changed using the appropriate switch commands.

```
\begin{namedsubeqs}{EQ}[style=\itshape\roman]
                \begin{namedsubeqs}{EQ}[style=\scshape]
```

$$A\mathbf{x} = \mathbf{b} \quad \text{(EQ.}i\text{)} \qquad A\mathbf{x} = \mathbf{b} \quad \text{(EQ.A)}$$
$$C\mathbf{y} = \mathbf{d} \quad \text{(EQ.}ii\text{)} \qquad C\mathbf{y} = \mathbf{d} \quad \text{(EQ.B)}$$

The `punct` argument controls the separator between the main tag and the subtag.

```
\begin{namedsubeqs}{EQ}[style=\scshape\roman, punct=:]

\begin{namedsubeqs}{EQ}[style=\bfseries\arabic, punct=$\sim$]
```

$$A\mathbf{x} = \mathbf{b} \quad \text{(EQ:I)} \qquad A\mathbf{x} = \mathbf{b} \quad \text{(EQ}\sim\mathbf{1}\text{)}$$
$$C\mathbf{y} = \mathbf{d} \quad \text{(EQ:II)} \qquad C\mathbf{y} = \mathbf{d} \quad \text{(EQ}\sim\mathbf{2}\text{)}$$

A single optional argument which does not contain an `=` sign is interpreted as the `style` argument. The default `style` and `punct` values may also be changed globally with `namedsubeqs-punct` and the post-load command `\setnamedsubeqstyle`; see Section 9.

```
\begin{namedsubeqs}{EQ}[\sffamily\Alph]
                \begin{namedsubeqs}{EQ}[\bfseries\arabic]
```

$$A\mathbf{x} = \mathbf{b} \qquad \text{(EQ.A)}$$
$$C\mathbf{y} = \mathbf{d} \qquad \text{(EQ.B)}$$

$$A\mathbf{x} = \mathbf{b} \qquad \text{(EQ.1)}$$
$$C\mathbf{y} = \mathbf{d} \qquad \text{(EQ.2)}$$

Tag/Subtag Overrides

The amsmath command `\tag` does not work in our IEEE-based `lp` environments. Instead, we have a few tag override commands: `\lptag` adjusts overrides the tag, `\lpsubtag` adjusts the subtag in a `namedsubeqs` environment and `\defaultsubtag` adds the default subtag format to a custom `\lptag`.

```
\begin{namedsubeqs}{EM}[\ttfamily\Alph]
\begin{lpregion}[qualifier-sep=0.2em]
  A\mathbf{x} &\GE \mathbf{b}
                    \lptag{m}\lpsubtag{1} \label{EM:1} \\
  \mathbf{a}_i^\top\mathbf{x} &\GE b_i
    &\forall\ i\in[\![n]\!]
                                          \label{EM:2} \\
  \sum_{j=1}^ma_{ij}x_j &\GE b_i
    &\forall\ i\in[\![n]\!]
                  \lptag{s}\defaultsubtag \label{EM:3} \\
\begin{bmatrix} ... \end{bmatrix}         \label{EM:4}
\end{lpregion} \end{namedsubeqs}

Referencing \eqref{EM:1} vs \eqref{EM:2} vs \eqref{EM:3}
vs \eqref{EM:4}.
```

$$A\mathbf{x} \geq \mathbf{b} \qquad \text{(m.1)}$$
$$\mathbf{a}_i^\top \mathbf{x} \geq b_i \qquad \forall\, i \in [\![n]\!] \qquad \text{(EM.A)}$$
$$\sum_{j=1}^{m} a_{ij}x_j \geq b_i \qquad \forall\, i \in [\![n]\!] \qquad \text{(s.B)}$$
$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \\ a_{m1} & a_{m2} & & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \geq \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \qquad \text{(EM.C)}$$

Referencing (m.1) vs (EM.A) vs (s.B) vs (EM.C).

6 Sub-References

Also available are `\lpsubref` and `\lpsubeqref`, which work like `\ref` and `\eqref` but reference only the secondary, sub-index part of a tag inside either `namedsubeqs` or a traditional `subequations` environment from the amsmath package.

```
  \begin{namedsubeqs}{SQ}[style=\itshape\roman]\label{nq}
  \begin{align}
    A\mathbf{x} = \mathbf{b}   \label{nq:1}\\
    C\mathbf{y} = \mathbf{d}   \label{nq:2}
  \end{align}
  \end{namedsubeqs}

Just like \ref{nq:1} and \eqref{nq:2}, we can reference
an equation's sub-tag \lpsubref{nq:1} and \lpsubeqref{nq:2}.
The environment itself may also be referenced \ref{nq}.
```

$$A\mathbf{x} = \mathbf{b} \tag{SQ.i}$$
$$C\mathbf{y} = \mathbf{d} \tag{SQ.ii}$$

Just like SQ.i and (SQ.ii), we can reference an equation's sub-tag i and (ii). The environment itself may also be referenced SQ.

`\lpsubref` and `\lpsubeqref` work with ordinary `\label` inside both `namedsubeqs` and traditional `subequations` environments. Other environments are not officially supported. If used on a label without a sub-tag, the main tag *should* be printed as normal.

```
    \begin{subequations}\label{sq}
    \renewcommand{\theequation}
              {\theparentequation\textbar\arabic{equation}}
    \begin{align}
      A\mathbf{x} = \mathbf{b}   \label{sq:1}\\
      C\mathbf{y} = \mathbf{d}   \label{sq:2}
    \end{align}
    \end{subequations}

We can also reference \ref{sq}, \ref{sq:1} and \eqref{sq:2},
or sub-reference \lpsubref{sq:1} and \lpsubeqref{sq:2} in a
\textsf{subequations} environment.
```

$$A\mathbf{x} = \mathbf{b} \tag{28|1}$$
$$C\mathbf{y} = \mathbf{d} \tag{28|2}$$

We can also reference 28, 28|1 and (28|2), or sub-reference 1 and (2) in a subequations environment.

Supported punctuation currently includes the period (.), colon (:), semicolon (;), comma (,), hyphen (-), forward slash (/), and `\textbar` (|). Other punctuation has not been tested.

7 Combining Macros

Each of these macros may be combined for a nicely named, labeled, formatted and tagged mathematical programming problem.

```
\paratitle{Non-Linear Program}[NLP]\label{para:NLP}
\begin{namedsubeqs}{NLP}[style=\itshape\roman]
\begin{lpalign}{Minimize}
  {f(\mathbf{x})   \lpsubtag{\textsc{obj}} \label{NLP.0}}
  g_i(\mathbf{x}) &\GE 0 &\forall\ i\in I  \label{NLP.1} \\
  h_j(\mathbf{x}) &\EQ 0 &\forall\ j\in J  \label{NLP.2} \\
      \mathbf{x}  &\GE 0                   \label{NLP.3}
\end{lpalign}
\end{namedsubeqs}

Problem \ref{para:NLP} is a nonlinear program.
Equation \eqref{NLP.0} is the objective function.
Equations (\ref{NLP.1}-\lpsubref{NLP.3}) are the constraints.
Non-negativity is enforced by constraint \lpsubeqref{NLP.3}.
```

Non-Linear Program (NLP)

$$\begin{aligned}
\text{Minimize} \quad & f(\mathbf{x}) && && \text{(NLP.OBJ)} \\
\text{s.t.} \quad & g_i(\mathbf{x}) \geq 0 && \forall\ i \in I && \text{(NLP.a)} \\
& h_j(\mathbf{x}) = 0 && \forall\ j \in J && \text{(NLP.b)} \\
& \mathbf{x} \geq 0 && && \text{(NLP.c)}
\end{aligned}$$

Problem NLP is a nonlinear program. Equation (NLP.OBJ) is the objective function. Equations (NLP.a-c) are the constraints. Non-negativity

is enforced by constraint (c).

8 Spacing Macros

Quick Spacing

Also included are font-relative versions of the common spacing commands `\quad` and `\qquad` and a few more for good measure:

- `\hskp` = 0.5em | |
- `\Hskp` = 1em | |
- `\HSkp` = 2em | |
- `\HSKp` = 3em | |
- `\HSKP` = 4em | |

Negative versions of the above spacing macros:

- `\htrg` = -0.5em
- `\Htrg` = -1em
- `\HTrg` = -2em
- `\HTRg` = -3em
- `\HTRG` = -4em

Vertical versions of the above horizontal spacing macros:

- `\vskp` = 0.5em
- `\Vskp` = 1em
- `\VSkp` = 2em
- `\VSKp` = 3em
- `\VSKP` = 4em
- `\vtrg` = -0.5em
- `\Vtrg` = -1em
- `\VTrg` = -2em
- `\VTRg` = -3em
- `\VTRG` = -4em

These are, I would say, less reliable than using `\vspace` or `\vskip` correctly, but I find them more convenient.

```
Some horizontal spacing between these \HSKP words

Some vertical spacing between these \VSKp lines

Some negative-horizontal spacing between these \HTrg words

Some negative-vertical space between these \Vtrg
lines, so that they overlap.
```

Some horizontal spacing between these words
Some vertical spacing between these

lines
Some negative-horizontal spacing between thesewords

Some negative vertical space between these lines, so that they overlap.

Delimiter Spacing

Some macros for tightening display-style operator spacing when the indexing is over-long. These preserve the usual subscript and superscript syntax while placing long limits in `\mathclap` boxes. They are meant for display math and are not generally useful inline.

- `\SUM` $= \sum$
- `\PROD` $= \prod$
- `\CUP` $= \bigcup$
- `\CAP` $= \bigcap$
- `\VEE` $= \bigvee$
- `\WEDGE` $= \bigwedge$

These operators now use the ordinary `_` and `^` syntax. This spacing is a personal preference; I think it makes things more consistent and legible. The example below is comically exaggerated; please use `\substack` to illuminate any *really* long indexing.

```
X=\sum_{i\in way too much indexing} x_i

                X=\VEE_{i\in way too much indexing} x_i

X=\CAP_{\substack{i\in way to \\ much indexing}} x_i

                  X=\PROD_{i=1,300,000}^{1,300,001} x_i
```

$$X = \sum_{i \in waytoomuchindexing} x_i \qquad X = \bigwedge_{i \in waytoomuchindexing} x_i$$

$$X = \bigcap_{\substack{i \in wayto \\ muchindexing}} x_i \qquad X = \prod_{i=1,300,000}^{1,300,001} x_i$$

Evaluator Spacing

A few binary relations with extra space on either side. I like these for aligned equations or inequality constraints.

- `\EQ` $=$
- `\LS` $<$
- `\GS` $>$
- `\IN` $\in$
- `\LE` $\leq$
- `\GE` $\geq$
- `\NEQ` $\neq$
- `\SUB` $\subset$
- `\SUP` $\supset$
- `\NIN` $\notin$
- `\SEQ` $\subseteq$
- `\SPQ` $\supseteq$

```
Ax+s &\eq b \\
 x,s &\ge 0
```

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax + s = b + 0 \\ & x, s \geq 0 \end{aligned}$$

```
Ax+s &\EQ b \\
   x,s &\GE 0
```

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax + s \;=\; b + 0 \\ & x, s \;\geq\; 0 \end{aligned}$$

The spacing defaults to 0.25em and is controlled by the package option `evaluator-spacing`; see Section 9.

9 Package Options

Default behaviors of many features can be changed with package-load options. Use the following keys in `\usepackage[<key>=<value>]{lpalign}`. For `st`, bare spaces in `\usepackage[...]` are stripped by LaTeX before this package sees the option value. Use `st={subject to}` or `st=subject~to` instead.

`row-sep` – controls the spacing between the rows. Defaults to `0.2em`. Section 2.

`objective-sep` – controls additional spacing between objective function and the first constraint. Defaults to `0em`.

`qualifier-sep` – controls the spacing between constraint body and the qualifier (for-all) term. Defaults to `2em`.

`gutter-sep` – controls the spacing between 'sense'/'subject to' terms and the constraint body within `lpalign`. Defaults to `0.75em`.

`st` – controls the default text in the 'subject to' gutter cell for `lpalign`. Defaults to `s.t.`.

`flush` – controls the default alignment behavior of constraints in the lpalign environment. Defaults to `a` but also accepts `l`, `c` and `r`.

`region-flush` – controls the default alignment behavior of constraints in the lpregion environment. Defaults to `a` but also accepts `l`, `c` and `r`. Section 3.

`namedpara-indent` – controls the indent of named paragraph markers. Defaults to `2em`. Section 4.

`namedpara-before` – controls the default material printed before a named paragraph abbreviation. Defaults to empty. Section 4.

`namedpara-after` – controls the default material printed after a named paragraph abbreviation. Defaults to empty.

`namedpara-flush` – controls the default alignment of named paragraph markers. Defaults to `left`.

`namedsubeqs-punct` – controls the default punctuation between the named par-

ent tag and the sub-index in `namedsubeqs`. Defaults to `..`

`evaluator-spacing` – controls the padding on either side of the spaced evaluators. Defaults to `0.25em`. Section 8.

Post-load commands are available for freer global style defaults:

`\setvarsstyle` – controls the global default value of `vars-style` in `lpalign`. Defaults to `\scriptstyle`. Good options include `\displaystyle` for larger variables and `\scriptscriptstyle` for smaller ones.

`\setnamedsubeqstyle` – controls the global default sub-index style used by `namedsubeqs`. Defaults to `\alph`. Section 6.

10 Version History

v1.0 Dec. 20, 2025. Initial distribution. Includes lpalign and namedsubeqs environments along with macros for named paragraphs and spacing.

v1.1 Jan. 30, 2026. Bug fix. Renamed quick spacing macros to maintain compatibility with standard TEX. Added negative, vertical and negative-vertical versions. Added summation spacing macros. Added clause alignment arguments to the lpalign environment.

v1.2 Jun. 25, 2026. Bug fix. Removed unused package dependencies, fixed the internal `\currentlabel` handling in namedsubeqs, added a no-hyperref fallback for `\lpsubref` and `\lpsubeqref`, added `\lpsubref` and `\lpsubeqref` support for the traditional subequations environment from amsmath.

v1.3 Jun. 26, 2026. Feature and compatibility update. Added `\namedpara` with key-value options, namespaced the sub-equation reference commands as `\lpsubref` and `\lpsubeqref`, and hardened parent-label handling for both `namedsubeqs` and traditional subequations.

v1.4 Jun. 28, 2026. Layout and defaults update. Reworked lpalign so the objective contributes correctly to the environment width, ordinary rows no longer need trailing `&` placeholders, package-load defaults are now available for `objective-sep`, `qualifier-sep` and `gutter-sep`, and the operator helpers now preserve standard `_/^` syntax.

v1.5 Jul. 6, 2026. Parser version of `lpalign` fixes s.t. alignment.

v1.6 Jul. 16, 2026. Rebuilt `lpalign` and `lpregion` on IEEEeqnarray, added centered and flushed layout modes, and unified package-load option names with the local hyphenated key style. Also added nestable `box` forms of the main environments.

v1.7 Jul. 17, 2026. Added `\lptag`, `\lpsubtag` and `\defaultsubtag` support for the IEEE backend, added post-load setters for global style defaults, and introduced a compact regression document for future smoke tests.

Contributions This package was developed by Jamie Fravel with assistance from OpenAI Codex from versions 1.2-1.7.